

Driver Drowsiness Detection — Implementation - Part 1

Project Title	Driver Drowsiness Detection System
Team Number	25
Team Members	Vangara Sridhar - 2023BCS0106 Suragouni Sahithi - 2023BCS0070 Vatte Ramya Sri - 2023BCS0208 Abhay Devanand Sawale - 2023BCS0046
Scope	Dataset collection, preprocessing, and approximately 25% methodology implementation
Date	17 September 2026

1. Project Overview

This project implements an end-to-end computer vision system for detecting driver drowsiness from webcam frames. The pipeline is camera capture -> Next.js frontend -> FastAPI backend -> YOLO classification -> JSON prediction -> visual status and alarm feedback.

Task 4 completion status

Requirement	Evidence	Status
Dataset collection	UTA-RLDD downloaded/configured through KaggleHub; documented in DATASET_INFO.md.	Completed
Pre-processing	Class mapping, reproducible split, YOLO directory generation, OpenCV decoding and RGB conversion.	Completed
Approx. 25% methodology	Webcam capture, upload, API routing, model loading, inference parsing, display and temporal alarm.	Implemented
Remaining work	Evaluation, direct eye-closure measurement, calibration, deployment hardening and final documentation.	Listed

2. Dataset Collection and Preparation

The project uses the UTA Real-Life Drowsiness Dataset (UTA-RLDD) from Kaggle. Source folders active and fatigue are mapped to Alert and Drowsy for the application.

Item	Recorded value
Total images	9,054
Distribution	Alert: 5,862 (~65%); Drowsy: 3,192 (~35%)
Generated split	Train: 6,337 Validation: 1,358 Test: 1,359
Class weights	Alert: 0.77 Drowsy: 1.42

Pre-processing implemented

Dataset discovery checks the KaggleHub cache and configured data directory. Image paths and labels are collected from class folders. Samples are split reproducibly with a fixed seed using a 70/15/15 ratio. A YOLO classification

directory is generated. Uploaded images are validated and decoded with OpenCV, converted from BGR to RGB, and passed to YOLO.

The source dataset is video-frame based. A future experiment should split by participant/video to reduce near-duplicate leakage between train and test sets.

3. Implemented Methodology

Stage	Module	Output
Capture	Camera.jsx	JPEG webcam frame
Transport	lib/api.js	multipart/form-data /predict
Decode	image_processing.py	Validated OpenCV image
Inference	model.py + inference.py	Class and confidence
Feedback	page.jsx + DetectionResult.jsx	Status, streak and alarm

Alarm behavior

Live monitoring sends approximately one frame per second. When the model returns Drowsy continuously, the frontend records the start time and activates the alarm after five seconds. An audio file can be placed at frontend/public/alarm.mp3; a browser tone is used as fallback. Alert results reset the timer.

Model safeguard

The backend validates checkpoint class names and rejects ImageNet-style models that lack Drowsy, Alert or Non_Drowsy. This prevents the earlier jersey/sunglasses failure mode.

Verification

Check	Result
npm run lint	Passed - no ESLint warnings
npm run build	Passed - Next.js build succeeded
Backend compile/import	Passed
GET /health	Passed - model_loaded=true
POST /predict	Passed - drowsiness-compatible response
CORS local-origin test	Passed

4. Major Tasks Remaining

Priority	Module	Next action
High	Formal evaluation	Report accuracy, precision, recall, F1, confusion matrix and calibration.
High	Eye-closure duration	Add face/eye landmarks or an eye-state model to measure closed eyes directly.
High	Reproducibility	Record dataset version, training command, checkpoint provenance and participant-level split.
Medium	Alarm UX	Add alarm.mp3 and test browser user-gesture/autoplay behavior.

Medium	Deployment	Configure production origins, Render/Vercel variables, logging and cold-start messaging.
Low	Analytics	Add session export, timestamps and privacy-preserving metrics.

5. Conclusion

Task 4 Part 1 is substantially implemented. Dataset integration and preprocessing are documented, and the first functional quarter of the methodology is demonstrated through the working webcam-to-inference pipeline. The next milestone is formal evaluation and a dedicated eye-closure/temporal model.

6. Code Implementation from the Project

The following excerpts are taken from the implemented repository and correspond to the modules described in the reference report. They show the working dataset-to-inference path and the temporal alarm behavior.

Code Snippet 1 - Webcam Frame Capture

Source: *frontend/components/Camera.jsx*

```
64   }, []);
65
66   // Capture a single frame and call onCapture
67   const captureFrame = useCallback(() => {
68     if (!videoRef.current || !canvasRef.current || !isReady) return;
69
70     const video = videoRef.current;
71     const canvas = canvasRef.current;
72
73     if (video.readyState < HTMLMediaElement.HAVE_CURRENT_DATA || !video.videoWidth || !video.videoHeight) {
74       return;
75     }
76
77     canvas.width = video.videoWidth;
78     canvas.height = video.videoHeight;
79
80     const ctx = canvas.getContext('2d');
81     if (!ctx) return;
82     ctx.drawImage(video, 0, 0, canvas.width, canvas.height);
83
84     canvas.toBlob((blob) => {
85       if (blob && onCapture) {
86         fpsCounterRef.current += 1;
```

Code Snippet 2 - Frontend API Request

Source: *frontend/lib/api.js*

```
28   }
29 }
30
31 /**
32  * Check backend health
33  * @returns {Promise<Object>} Health status
34  */
35 export async function checkHealth() {
36   const response = await fetchWithTimeout(getApiUrl('/health'));
37
38   if (!response.ok) {
39     throw new Error('Health check failed: ${response.statusText}`);
40   }
41
42   return response.json();
43 }
44
45 /**
46  * Send image for drowsiness prediction
47  * @param {Blob|File} imageBlob - Image to analyze
48  * @returns {Promise<Object>} Prediction results
49  */
50 export async function predictDrowsiness(imageBlob) {
51   if (!(imageBlob instanceof Blob) || imageBlob.size === 0) {
52     throw new Error('Could not capture a valid image. Please try again.');
```

Code Snippet 3 - Image Validation and Prediction Route

Source: *backend/app/routes/predict.py*

```
24   Args:
25     file: Uploaded image file (JPEG, PNG)
26
27   Returns:
28     JSON with prediction results:
29     {
30       "label": "Drowsy" or "Non_Drowsy",
31       "confidence": float (0-1),
32       "status": "drowsy" or "alert",
33       "all_probs": {class_name: probability}
34     }
```

```

35     """
36     # Validate content type
37     if file.content_type not in ["image/jpeg", "image/jpg", "image/png", "image/webp"]:
38         raise HTTPException(
39             status_code=400,
40             detail=f"Invalid file type: {file.content_type}. Only JPEG, PNG, and WebP are supported."
41         )
42
43     try:
44         # Read file bytes
45         file_bytes = await file.read()
46
47         if len(file_bytes) > MAX_IMAGE_BYTES:
48             raise HTTPException(
49                 status_code=413,
50                 detail=f"Image is too large. Maximum size is {MAX_IMAGE_BYTES // (1024 * 1024)} MB."
51             )
52
53         # Validate image
54         if not validate_image(file_bytes):
55             raise HTTPException(
56                 status_code=400,
57                 detail="Invalid image file. Could not decode image."
58             )
59
60         # Decode image
61         image = decode_image(file_bytes)
62
63         # Run inference
64         prediction = predict_drowsiness(image, conf_threshold=0.25)
65
66         return JSONResponse(
67             status_code=200,
68             content={
69                 "success": True,
70                 "prediction": prediction
71             }
72         )
73
74     except HTTPException:
75         raise
76     except Exception as e:
77         # Log error
78         print(f"Error during prediction: {type(e).__name__}: {e}", file=sys.stderr)
79
80         raise HTTPException(
81             status_code=500,
82             detail="Prediction failed. Please try again."

```

Code Snippet 4 - Model Loading and Class Safeguard

Source: *backend/app/models/model.py*

```

25     """
26     def __init__(self, model_path: str):
27         """
28         Initialize YOLO model
29
30         Args:
31             model_path: Path to YOLO weights file (.pt)
32         """
33         path = Path(model_path)
34         if not path.is_absolute():
35             path = Path(__file__).resolve().parents[2] / path
36         if not path.is_file():
37             raise FileNotFoundError(f"Model weights not found at {path}")
38
39         # Load YOLO model
40         self.model = YOLO(str(path))
41
42         class_names = {str(name).lower().replace("-", "_") for name in self.model.names.values()}
43         if not ({"drowsy", "alert", "non_drowsy"} & class_names):
44             raise ValueError(
45                 "The configured model is not a drowsiness classifier. "
46                 "Expected a Drowsy/Alert/Non_Drowsy class."
47             )
48
49         # Set device (use GPU if available)
50         self.device = 'cuda' if torch.cuda.is_available() else 'cpu'
51         self.model.to(self.device)
52
53         print(f"✓ YOLO model loaded from {path}")
54         print(f"✓ Using device: {self.device}")

```

Code Snippet 5 - Five Second Drowsiness Alarm

Source: *frontend/app/page.jsx*

```

76
77 // Stop alarm when live mode is turned off
78 useEffect(() => {
79   if (!isLive) stopAlarm();
80 }, [isLive, stopAlarm]);
81
82 // Cleanup alarm on unmount
83 useEffect(() => () => stopAlarm(), [stopAlarm]);
84
85 // ----- Frame handler -----
86 const handleCapture = useCallback(async (imageBlob) => {
87   // Skip if already waiting for a response (live mode)
88   if (isProcessingRef.current) return;
89   isProcessingRef.current = true;
90   setIsLoading(true);
91   setError(null);
92
93   try {
94     const result = await predictDrowsiness(imageBlob);
95
96     if (result.success && result.prediction) {
97       const pred = result.prediction;
98       setPrediction(pred);
99       setBackendStatus('connected');
100
101       const drowsy = pred.status === 'drowsy';
102
103       setSessionStats(prev => ({
104         total: prev.total + 1,
105         drowsy: prev.drowsy + (drowsy ? 1 : 0),
106         alert: prev.alert + (drowsy ? 0 : 1),
107       }));
108
109       if (drowsy) {
110         const now = Date.now();
111         if (drowsySinceRef.current === null) drowsySinceRef.current = now;
112         const elapsedSeconds = (now - drowsySinceRef.current) / 1000;
113         setDrowsyStreak(Math.floor(elapsedSeconds));
114         if (elapsedSeconds >= DROWSY_ALARM_SECONDS) startAlarm();
115       } else {
116         drowsySinceRef.current = null;
117         setDrowsyStreak(0);
118         stopAlarm();
119       }

```

Implementation note: place the optional alarm audio at *frontend/public/alarm.mp3*. Live monitoring evaluates roughly one frame per second and starts the alarm after five seconds of continuous Drowsy predictions.